

PROGRAMAÇÃO FUNCIONAL

INTRODUÇÃO A HASKELL

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLOGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- Compreender a sintaxe fundamental para declaração de funções puras.
- Analisar o sistema de tipos da linguagem (estático e forte).
- Compreender o processo de verificação de tipos em tempo de compilação.

CARACTERÍSTICAS DE HASKELL

- Recursão ao invés de iteração.
- Casamento de padrão (muitas linguagens modernas possuem suporte, como Rust e Python).

CARACTERÍSTICAS DE HASKELL

- Em Haskell, a computação é guiada pela **avaliação** de expressões **matemáticas**.
- **Funções** são **puras**: dado o mesmo conjunto de argumentos, o retorno será sempre invariável.
- **Não** há conceito de estado mutável ou **variáveis** tradicionais.

CARACTERÍSTICAS DE HASKELL

- Funções são cidadãos de primeira **classe**.
 - Podem ser passadas como argumento para outras funções.
 - Podem ser retornadas como resultado de outras funções.
 - Podem ser criadas localmente (funções anônimas).

RECURSÃO

Iteração: repetir um processo um certo número de vezes.

```
1.int factorial = 1;  
2.for (int i = 1 ; i <= n ; i++)  
3.    factorial *= i;  
4.printf("Fatorial: %d" , sum);
```

RECURSÃO

Recursão: definir algo em termos de si próprio.

1. $\text{factorial} :: \text{Int} \rightarrow \text{Int}$

2. $\text{factorial } 0 = 1$

3. $\text{factorial } n = n * \text{factorial } (n - 1)$

CASAMENTO DE PADRÃO

- Funções são definidas por uma sequência de equações.
 - Argumentos são comparados com o lado esquerdo até que um dos lados "case".

CASAMENTO DE PADRÃO

Na definição de fatorial ...

- Se o argumento for **0**, o valor retornado é **0**.
- Caso contrário, a variável **n** casa com o valor do argumento e esse é substituído no lado direito da equação.

FATORIAL

$$0! = 1$$

$$1! = 1 * 0!$$

$$2! = 2 * 1!$$

$$3! = 3 * 2!$$

$$4! = 4 * 3!$$

$$5! = 5 * 4!$$

$$6! = 6 * 5!$$

....

$$n! = n * (n-1)!$$

EXPRESSÕES

Em linguagens imperativas (C, C++, Python) comandos manipulam o **estado** do programa.

- Estado: **variáveis** visíveis em um dado ponto do código.
- Comandos são executados **sequencialmente**.
- Variáveis dão nome a uma parte do estado do programa.

EXPRESSÕES

Em linguagens funcionais Expressões representam um **valor** que depende apenas de **sub-expressões**.

- Isso facilita a composição e raciocínio sobre programas.

EXEMPLO

Considere a função a seguir:

1. $\text{fact} :: \text{Int} \rightarrow \text{Int}$

2. $\text{fact } n = n * \text{fact } (n - 1)$

3. $\text{fact } 0 = 1$

Qual o Fatorial de 5?

ESTATICAMENTE TIPADA

- Toda expressão e função possui um tipo.
- O compilador evita **combinações** erradas.

ESTATICAMENTE TIPADA

Mas é necessário anotar tipos em todas as definições?

- Anotar tipos é uma boa prática.
- Mas o compilador de Haskell é capaz de inferir o tipo da expressão.

LINGUAGEM PURA

- Em Haskell, variáveis **não** podem ter valores modificados.
 - Variáveis apenas dão **nome** a valores.
- Efeitos **colaterais** são identificadas por seu tipo.
 - Exemplo: toda função que faz I/O possui o tipo IO.

INTERPRETADOR

EXEMPLOS

1.length [1,2,3]

1.sum [1 .. 10]

1.reverse [1 .. 10]

INTERPRETADOR

EXEMPLOS

Parêntesis servem para separar subexpressões.

1.sum (replicate 3 1): um argumento para sum.

1.sum replicate 3 1: três argumentos para sum.

O primeiro exemplo é aceito, o segundo é um erro de tipo pois sum recebe somente um argumento.

DEFININDO FUNÇÕES

- Nomes de funções e argumentos em letras minúsculas.
- Chamada de função não usa parêntesis.

DEFININDO FUNÇÕES

Tipos de funções são representados em Haskell da seguinte forma:

$$A \rightarrow B$$

- A é o tipo(s) dos argumentos.
- B é o tipo do resultado.

DEFININDO FUNÇÕES

Exemplo:

1. $\text{add1} :: \text{Int} \rightarrow \text{Int}$

2. $\text{add1 } x = x + 1$

`add` é o nome da função, ela tem como argumento/parâmetro um Inteiro e como resultado um inteiro.

DEFININDO FUNÇÕES

Exemplo: função com dois argumentos.

1. $\text{media} :: \text{Float} \rightarrow \text{Float} \rightarrow \text{Float}$

2. $\text{media } x \ y = (x + y) / 2$

DEFININDO FUNÇÕES

Exemplo: Defina uma função que receba 3 números inteiros e retorne sua soma.

1. $\text{sum3} :: \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

2. $\text{sum3 } x \ y \ z = x + y + z$

LISTAS

Estrutura de dados **fundamental** em programação funcional.

- Listas possuem dois padrões básicos
 - $[]$ denota a lista vazia.
 - $x : xs$ denota a lista com cabeça(primeiro elemento) x e cauda xs (demais elementos).

Chamei de x e xs , mas poderia ser qualquer nome.

LISTAS

Listas são tão comuns em Haskell que a linguagem possui sintaxe especial para representá-las.

Exemplo:

- $1 : (2 : (3 : []))$ pode ser escrita como $[1,2,3]$.

`:"` é o operador de inserção na lista

LISTAS

Exemplos:

- $1 : [5]$ -- Elemento 1 inserido na lista que contem o elemento 5
- $2 : (4 : [1, 6])$ -- Elemento 2 inserido na lista resultante de inserir o elemento 4 na lista contendo $[1, 6]$;

LISTAS

Como Haskell é uma linguagem **fortemente** tipadas, listas possuem **tipos**.

O tipo **[a]** denota listas cujos elementos são do tipo **a**.

- Listas são **polimórficas**.
- O **a** em **[a]** denota uma **variável de tipo**, que são universalmente quantificadas, ou seja é indiferente ao tipo.

LISTAS

Construtor de lista:

- $[] :: []$
- $(:) :: a \rightarrow [a] \rightarrow [a]$

LISTAS

Algumas funções sobre listas:

- **null** testa se uma lista é ou não vazia.
- **head** retorna o primeiro elemento de uma lista.
- **tail** retorna a cauda de uma lista.

head e **tail** resultam em erro se chamadas para a lista vazia.

LISTAS

Quais os tipos das funções **null**, **head** e **tail**?

- A função **null** verifica se uma lista é **vazia** ou não
 - $\text{null} :: [a] \rightarrow \text{Bool}$
- A função **head** retorna o primeiro **elemento** de uma **lista**.
 - $\text{head} :: [a] \rightarrow a$
- A função **tail** retorna a **cauda** de uma **lista**.
 - $\text{tail} :: [a] \rightarrow [a]$

LISTAS

Por que as assinaturas das funções **null**, **head** e **tail** utilizam variáveis de tipo **genéricas** (ex: **a**)?

- **Independência de Conteúdo**: Para verificar se uma lista está **vazia** ($\text{null} :: [a] \rightarrow \text{Bool}$), a função inspeciona estritamente a **estrutura do dado** (o **construtor []**). A natureza dos elementos armazenados é **irrelevante** para essa verificação.

LISTAS

Por que as assinaturas das funções **null**, **head** e **tail** utilizam variáveis de tipo **genéricas** (ex: **a**)?

- **Acesso Estrutural:** Para extrair o **primeiro** elemento ($\text{head} :: [a] \rightarrow a$) ou a **cauda** da lista ($\text{tail} :: [a] \rightarrow [a]$), a operação computacional **não requer conhecimento** prévio sobre o **tipo** concreto do dado sendo manipulado.

CASAMENTO DE PADRÃO EM LISTAS

O mecanismo de **casamento de padrão** permite a inspeção da **estrutura** de uma lista através de seus **construtores fundamentais**: a **lista vazia** (`[]`) e o operador **cons** (`:`).

CASAMENTO DE PADRÃO EM LISTAS

A extração de dados pode ser realizada de maneira **estática**, limitando-se a um **número finito de elementos**, sem a necessidade de **indução** ou **iteração**.

CASAMENTO DE PADRÃO EM LISTAS

Exemplos Práticos:

-- Avaliação estrutural finita

`analisaTamanho :: [a] -> String`

`analisaTamanho []` = "A lista esta vazia."

`analisaTamanho [x]` = "A lista possui exatamente um elemento."

`analisaTamanho (x:y:xs)` = "A lista possui multiplos elementos."

CASAMENTO DE PADRÃO EM LISTAS

Exemplos Práticos:

-- Extração de valores e uso do padrão curinga (`_`)

`somaDoisPrimeiros :: [ Int ] -> Int`

`somaDoisPrimeiros (x:y:_) = x + y`

`somaDoisPrimeiros _ = 0`

EXERCICIOS

Exercício 1: Cálculo Matemático Simples

- Objetivo: Calcular a área de um retângulo dadas a sua base e altura.

1. `areaRetangulo :: Double -> Double ->`

2. `DoubleareaRetangulo = undefined`

EXERCICIOS

Exercício 2: Média de Valores

- Objetivo: Calcular a média aritmética de três valores em ponto flutuante.

1. mediaTresValores :: Double -> Double -> Double
-> Double

2. mediaTresValores = undefined

EXERCICIOS

Exercício 3: Acesso Posicional em Listas Estáticas

- Objetivo: Utilizar pattern matching para retornar o terceiro elemento de uma lista numérica. Tratar listas com menos de 3 elementos retornando 0.

1.terceiroElemento :: Num a => [a] -> a

2.terceiroElemento = undefined

EXERCICIOS

Exercício 4: Validação de Padrão Literal

- Objetivo: Verificar se o primeiro elemento de uma lista de inteiros é estritamente o valor zero, utilizando o casamento direto de valor.

1. `comecaComZero :: [Int] -> Bool`

2. `comecaComZero = undefined`

3. `comecaComZero = undefined`

EXERCICIOS

Exercício 5: Extração e Operação Algébrica

- Objetivo: Somar os três primeiros elementos de uma lista numérica. Caso a lista possua menos de três elementos, a função deve retornar 0.

1. `somaTresPrimeiros :: Num a => [a] -> a`

2. `somaTresPrimeiros = undefined`

DÚVIDAS???